Intro

What is Machine Learning:

- Machine Learning is a subdomain of computer science that focuses on algorithms which help a computer learn from data without **explicit** programming

AI vs ML vs DS:

Artificial Intelligence is an area of computer science, where the goal is to enable computers and machines to perform human like tasks and simulate human behaviour

Machine Learning is a subset of AI that tries to solve a specific problem and make predictions using data

Data Science is a field that attempts to find patterns and draw insights from data(might use ML)

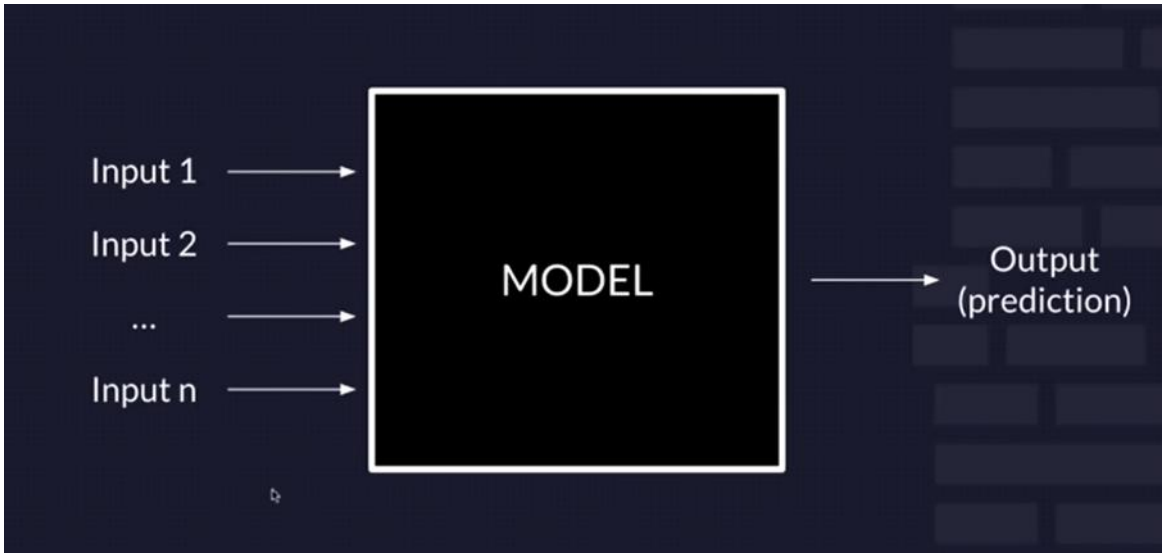
Types of Machine Learning:

Supervised Learning: Uses labeled inputs to train models and learn outputs
Ex: Training a model to predict if an image is a cat or a dog

Unsupervised Learning: Uses unlabeled data to learn patterns in data
Ex: If given bunch of cat and dogs images without labels, model is able to cluster cats and dogs

Reinforcement Learning: Agent learning in interactive environment based on rewards and penalties
Ex: Training a computer algorithm by giving rewards if it does things that we need it to

Supervised Learning:



Input 1, Input 2 etc are called features

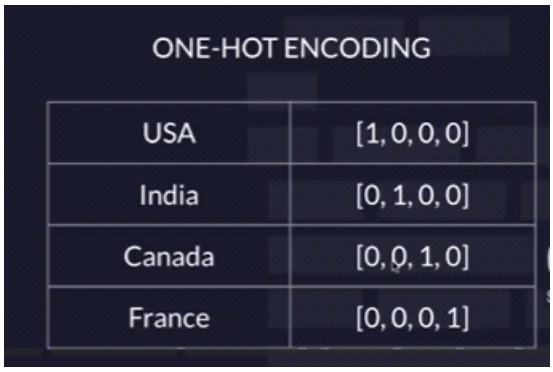
Features:

- **Qualitative:** Categorical data (Finite number of categories or groups)

Ex: Gender, Country

There is no inherent order in the above type of data. So, this is also called Nominal data

We use ONE-HOT Encoding to feed this data into computer



There can also data where there is inherent order. This is ordinal data. Examples: Age groups, ratings etc. We give a number to each category and feed it to computer.

- **Quantitative:** Numerical valued data (could be discrete or continuous)
  Ex: Length, Temperature, Number of chocolates etc.

  Length and Temperature are continuous quantitative data and Number of Chocolates is Discrete Quantitative data.

These features are fed into training the model.

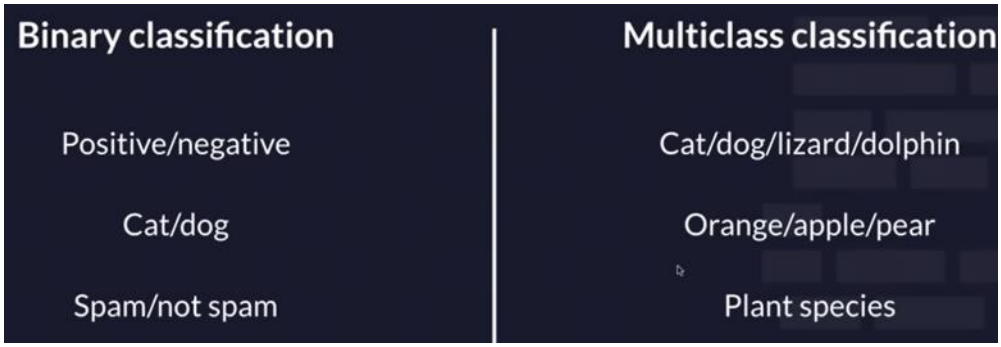
Output of the model:

- **Classification:** Predict discrete classes

This is multiclass classification.

There is also Binary classification. Only two categories.

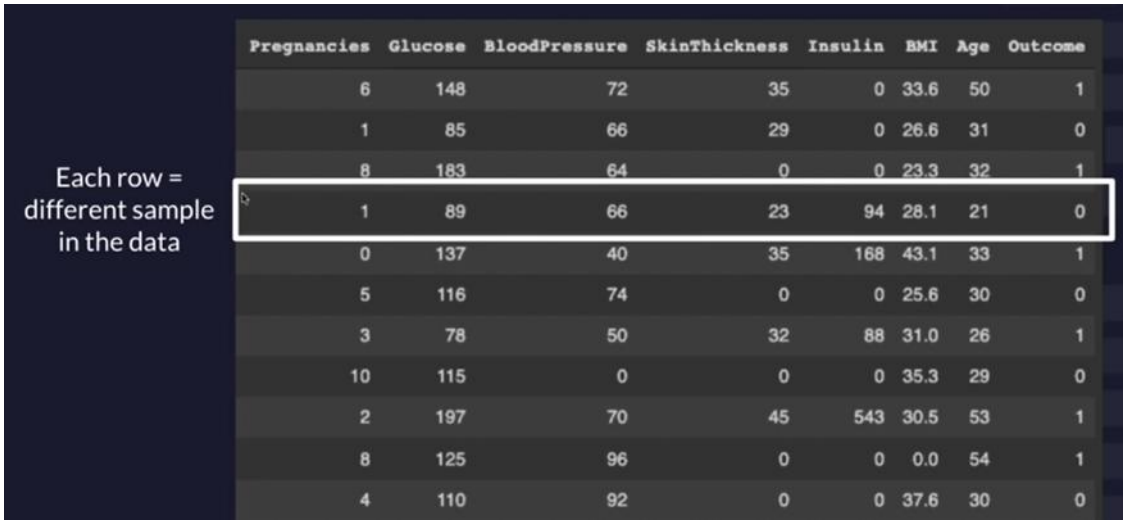
Examples:



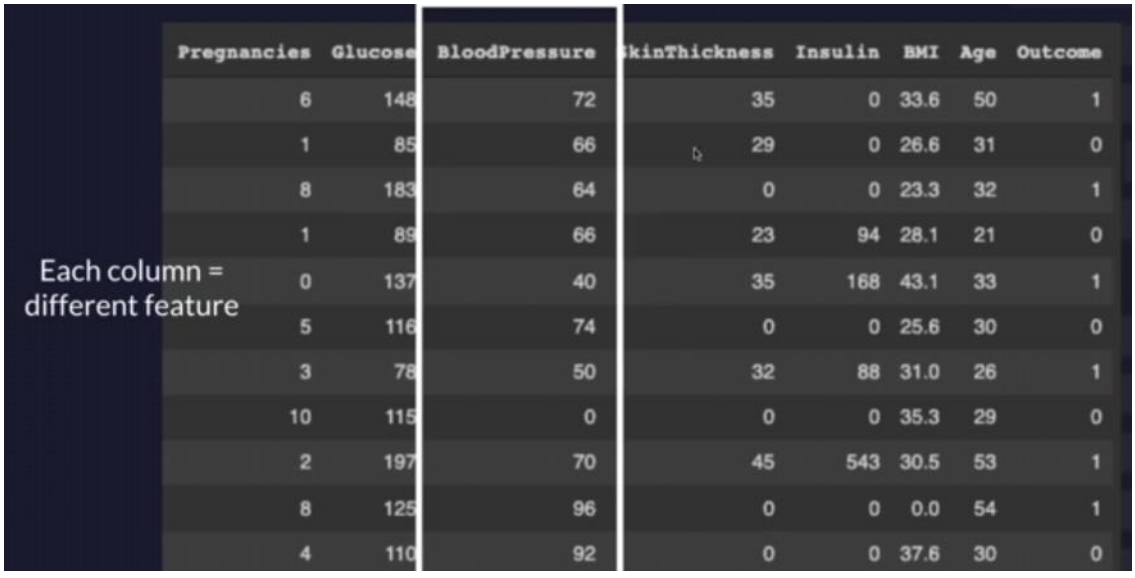
- **Regression:** Predict continuous values
  Examples: Price of Etherium, Price of a house, Temperature

  We are trying to predict a number which is close to true value from different features of the dataset.

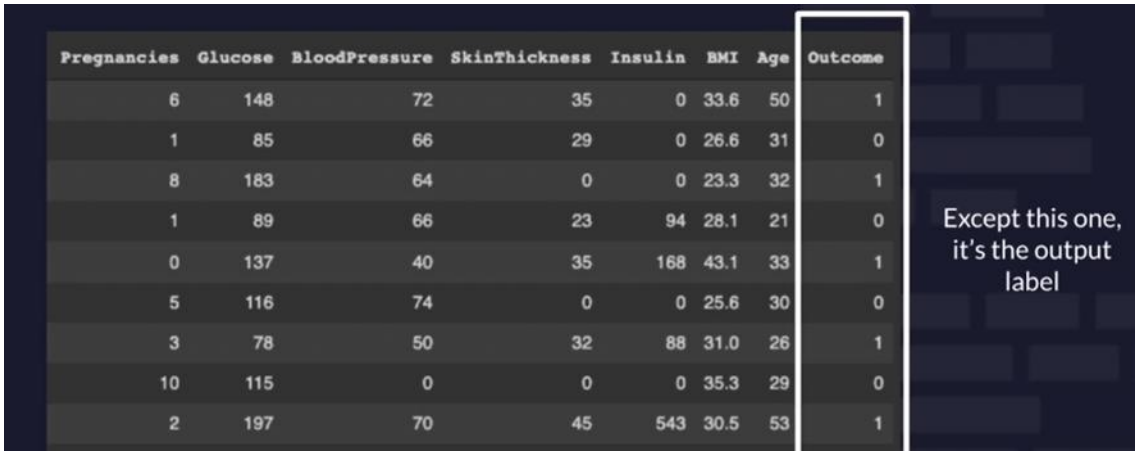
Supervised Learning Dataset:



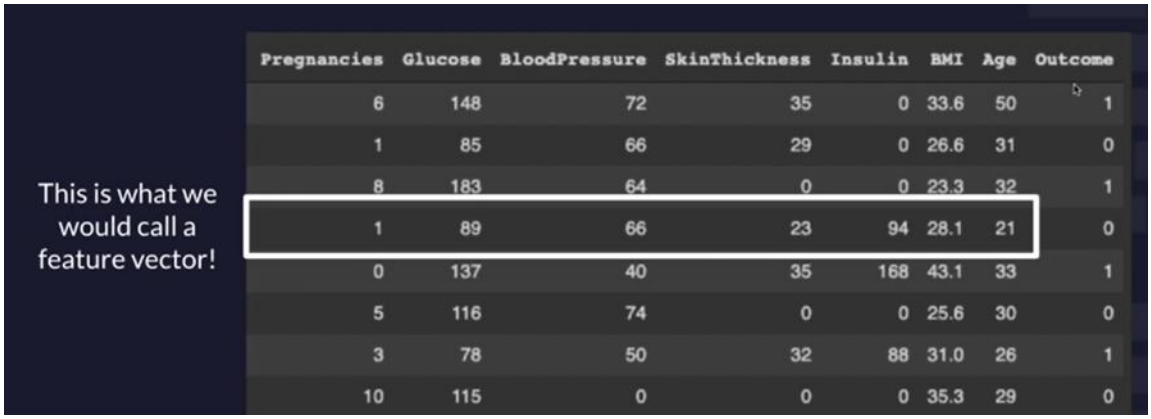
Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	Age	Outcome
6	148	72	35	0	33.6	50	1
1	85	66	29	0	26.6	31	0
8	183	64	0	0	23.3	32	1
1	89	66	23	94	28.1	21	0
0	137	40	35	168	43.1	33	1
5	116	74	0	0	25.6	30	0
3	78	50	32	88	31.0	26	1
10	115	0	0	0	35.3	29	0
2	197	70	45	543	30.5	53	1
8	125	96	0	0	0.0	54	1
4	110	92	0	0	37.6	30	0



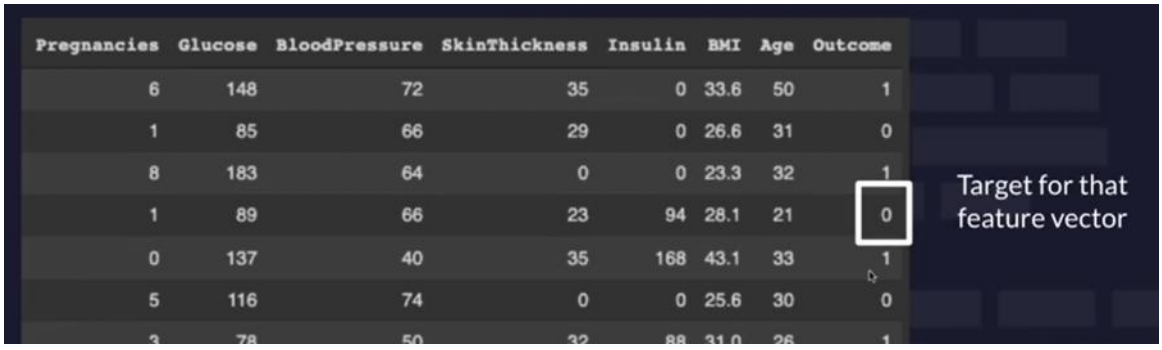
Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	Age	Outcome
6	148	72	35	0	33.6	50	1
1	85	66	29	0	26.6	31	0
8	183	64	0	0	23.3	32	1
1	89	66	23	94	28.1	21	0
0	137	40	35	168	43.1	33	1
5	116	74	0	0	25.6	30	0
3	78	50	32	88	31.0	26	1
10	115	0	0	0	35.3	29	0
2	197	70	45	543	30.5	53	1
8	125	96	0	0	0.0	54	1
4	110	92	0	0	37.6	30	0



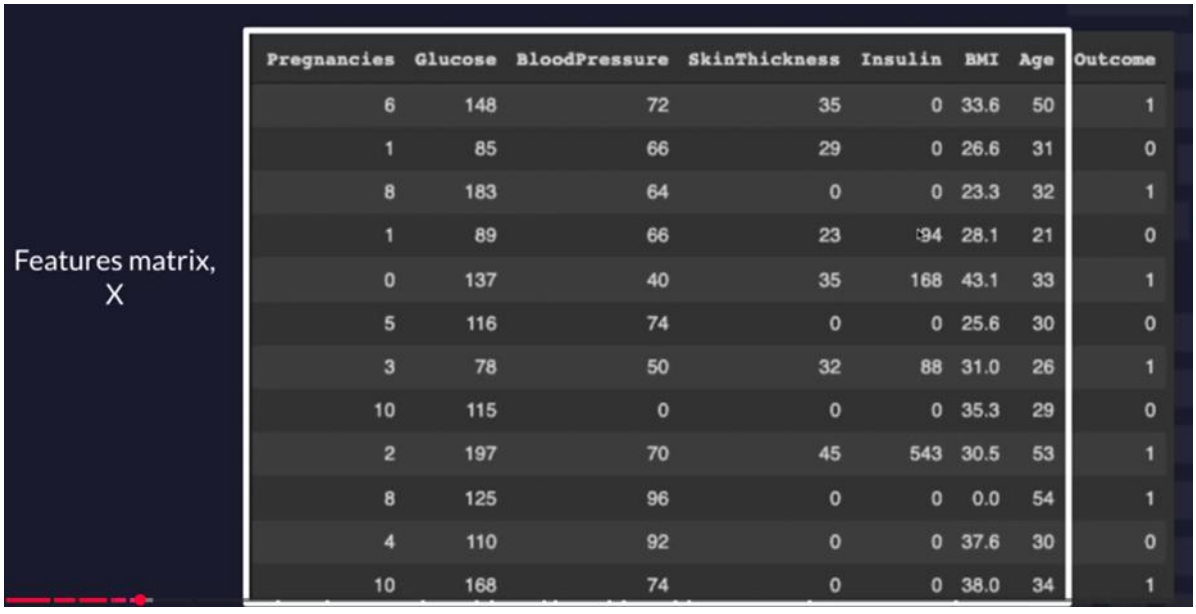
Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	Age	Outcome
6	148	72	35	0	33.6	50	1
1	85	66	29	0	26.6	31	0
8	183	64	0	0	23.3	32	1
1	89	66	23	94	28.1	21	0
0	137	40	35	168	43.1	33	1
5	116	74	0	0	25.6	30	0
3	78	50	32	88	31.0	26	1
10	115	0	0	0	35.3	29	0
2	197	70	45	543	30.5	53	1



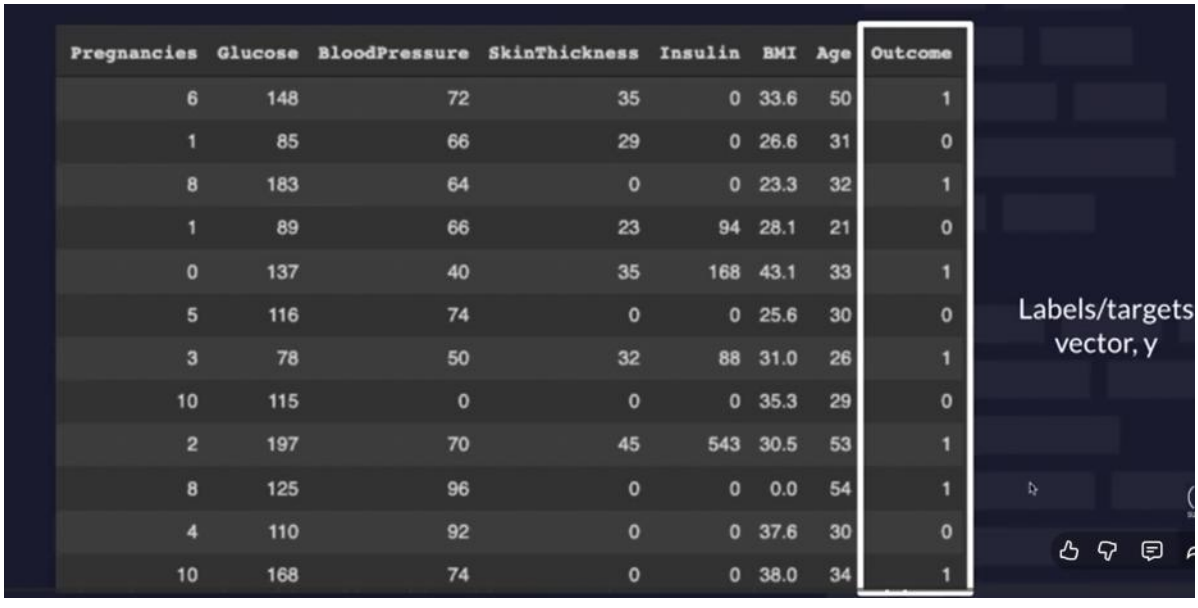
Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	Age	Outcome
6	148	72	35	0	33.6	50	1
1	85	66	29	0	26.6	31	0
8	183	64	0	0	23.3	32	1
1	89	66	23	94	28.1	21	0
0	137	40	35	168	43.1	33	1
5	116	74	0	0	25.6	30	0
3	78	50	32	88	31.0	26	1
10	115	0	0	0	35.3	29	0



Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	Age	Outcome
6	148	72	35	0	33.6	50	1
1	85	66	29	0	26.6	31	0
8	183	64	0	0	23.3	32	1
1	89	66	23	94	28.1	21	0
0	137	40	35	168	43.1	33	1
5	116	74	0	0	25.6	30	0



Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	Age	Outcome
6	148	72	35	0	33.6	50	1
1	85	66	29	0	26.6	31	0
8	183	64	0	0	23.3	32	1
1	89	66	23	94	28.1	21	0
0	137	40	35	168	43.1	33	1
5	116	74	0	0	25.6	30	0
3	78	50	32	88	31.0	26	1
10	115	0	0	0	35.3	29	0
2	197	70	45	543	30.5	53	1
8	125	96	0	0	0.0	54	1
4	110	92	0	0	37.6	30	0
10	168	74	0	0	38.0	34	1



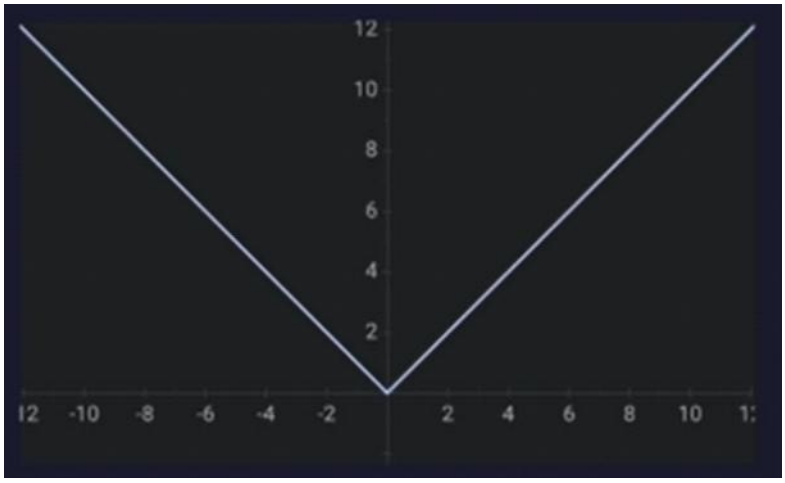
Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	Age	Outcome
6	148	72	35	0	33.6	50	1
1	85	66	29	0	26.6	31	0
8	183	64	0	0	23.3	32	1
1	89	66	23	94	28.1	21	0
0	137	40	35	168	43.1	33	1
5	116	74	0	0	25.6	30	0
3	78	50	32	88	31.0	26	1
10	115	0	0	0	35.3	29	0
2	197	70	45	543	30.5	53	1
8	125	96	0	0	0.0	54	1
4	110	92	0	0	37.6	30	0
10	168	74	0	0	38.0	34	1

Metrics of Performance:

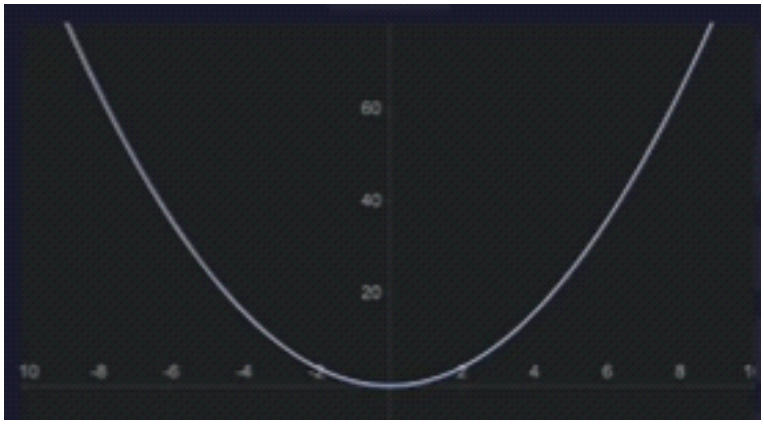
Loss: Loss is difference between your prediction and actual label

Loss Functions:

- **L1 Loss:** loss = sum(|y_{real} - $y_{predicted}$|)

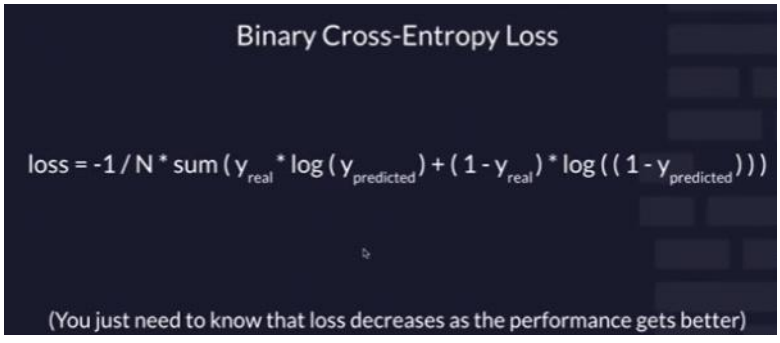


- **L2 Loss:** sum((|y_{real} - $y_{predicted}$|)^2)



If the predicted value is close to real value, penalty is less, but if it is far, penalty is high. The loss function is quadratic.

- **Binary Cross Entropy Loss:**



Accuracy: The metric which tells how well the model predicted the label.

k-Nearest Neighbors (kNN) Algorithm:

What is kNN?

Imagine you just moved to a new neighborhood and want to figure out if you'll like living there. What would you do? You'd probably look at your neighbors! If most people around you are friendly, active, and similar to you, chances are you'll enjoy the neighborhood too. This is exactly how the k-Nearest Neighbors algorithm works.

kNN is one of the simplest yet most powerful algorithms in machine learning, developed back in 1951 by Evelyn Fix and Joseph Hodges. The fundamental principle is beautifully intuitive: **things that are similar tend to be close together, and you can predict something about a new item by looking at items similar to it**.

How Does kNN Work? A Step-by-Step Journey

Flowchart illustrating the complete step-by-step process of the k-Nearest Neighbors algorithm, from loading data to making predictions for both classification and regression tasks

Let me walk you through how kNN makes predictions using a simple example. Suppose you run a fruit store and want to automatically classify fruits based on their weight and color intensity. You have historical data about apples and oranges, and a new fruit arrives that you need to classify.

Step 1: Choose the Value of k

The first critical decision is selecting **k**, which represents how many "neighbors" (similar examples) you want to consult. Think of k as asking "How many friends should I poll before making a decision?". If k=5, you're asking the 5 most similar fruits for their opinion.

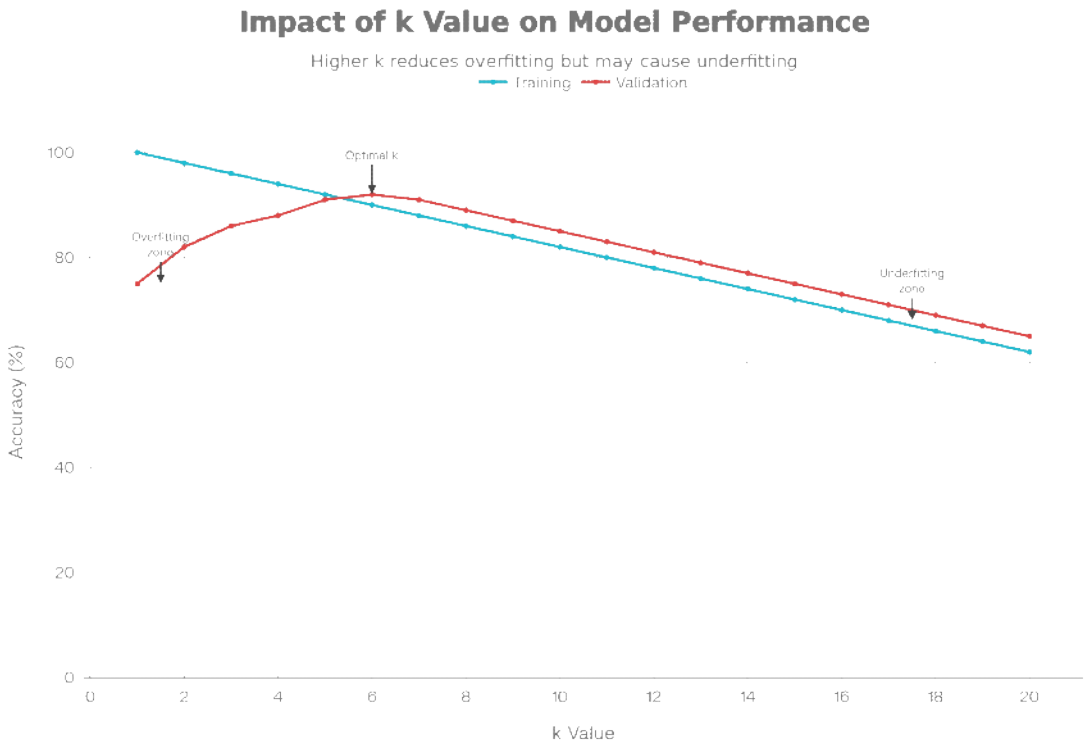
Choosing k is crucial because it directly impacts your results:

- **Small k values (like k=1 or k=3)**: You're only consulting your closest neighbors, which makes you very sensitive to individual quirks or noise in the

data. It's like making a life decision based on advice from just one friend—you might get misled if that friend is having a bad day.

- **Large k values (like k=15 or k=20)**: You're consulting many neighbors, which makes your decision more stable but potentially too generalized. It's like asking 20 people for restaurant recommendations—you'll probably get a safe, popular choice, but might miss hidden gems.

The relationship between k value and model accuracy. Small k values lead to overfitting (high training accuracy but lower validation accuracy), while large k values cause underfitting. The optimal k typically falls in the middle range (k=5-7 in this example)

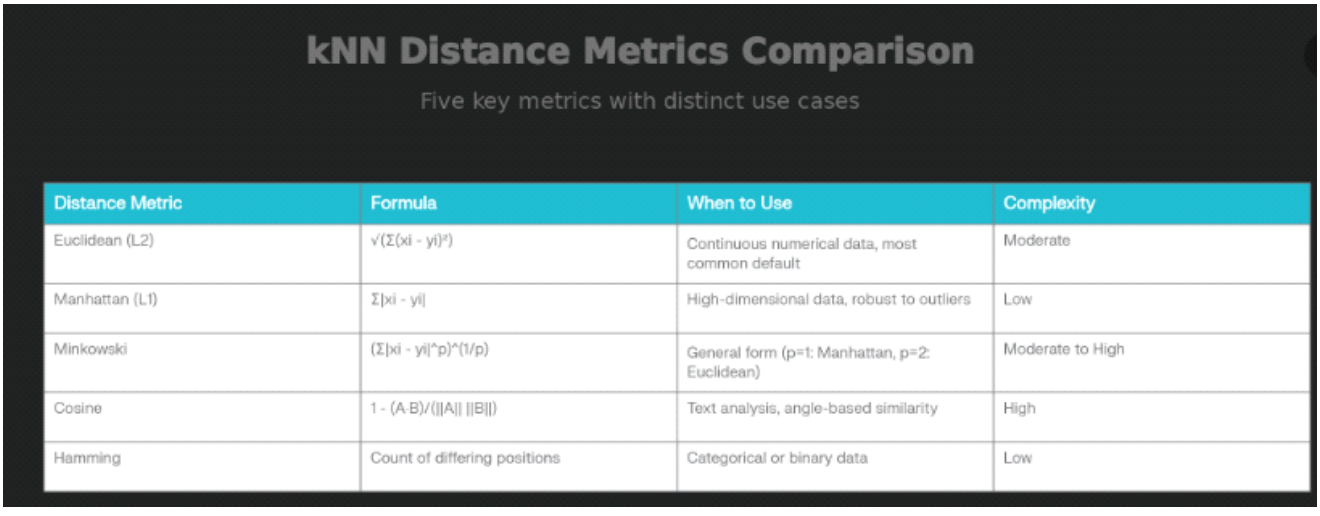


The sweet spot typically falls between k=3 and k=7 for most problems. We use techniques like **cross-validation** to find the optimal k: we try different k values, measure how well each performs, and pick the one that gives the best results

Step 2: Calculate Distances

Once you have your k value, the algorithm needs to measure how "similar" or "close" the new fruit is to each fruit in your training data. This is where **distance metrics** come in—mathematical formulas that quantify similarity.

Comparison of common distance metrics used in kNN algorithm, showing their formulas, appropriate use cases, and computational complexity



Distance Metric	Formula	When to Use	Complexity
Euclidean (L2)	√(Σ(xi - yi)²)	Continuous numerical data, most common default	Moderate
Manhattan (L1)	Σ|xi - yi|	High-dimensional data, robust to outliers	Low
Minkowski	(Σ|xi - yi|^p)^(1/p)	General form (p=1: Manhattan, p=2: Euclidean)	Moderate to High
Cosine	1 - (A·B)/(||A|| ||B||)	Text analysis, angle-based similarity	High
Hamming	Count of differing positions	Categorical or binary data	Low

Euclidean Distance: The Straight-Line Distance

This is the most common distance measure, and it calculates the straight-line

distance between two points, just like measuring the distance between two houses on a map

The Math (explained simply):
Imagine you have two fruits: Fruit A at position (weight=150g, color=7) and Fruit B at position (weight=180g, color=5). To find the distance:

a. Find the difference in each feature:
   - Weight difference: 180 - 150 = 30
   - Color difference: 5 - 7 = -2
b. Square each difference (this makes negative numbers positive and emphasizes larger differences):
   - Weight: $30^2 = 900$
   - Color: $(-2)^2 = 4$
c. Add them up: 900 + 4 = 904
d. Take the square root: $\sqrt{904} \approx 30.07$

Formula: $d = \sqrt{[(x_2 - x_1)^2 + (y_2 - y_1)^2]}$
For more dimensions (features), you simply add more squared differences: $\sqrt{[(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2 + ...]}$

This is also called the **L2 norm** or L2 distance

Manhattan Distance: The City Block Distance

Named after the grid layout of Manhattan streets, this distance measures how far you'd walk if you could only move along a grid (like walking city blocks)

The Math (explained simply):
Using the same two fruits from before:
a. Find absolute differences: |180 - 150| = 30, |5 - 7| = 2
b. Add them up: 30 + 2 = 32
That's it! Much simpler than Euclidean distance.
Formula: $d = |x_2 - x_1| + |y_2 - y_1|$

Manhattan distance is more robust to outliers (extreme values) and often works better when you have many features (high-dimensional data). Studies show Manhattan distance frequently outperforms other metrics in kNN applications
This is also called the **L1 norm** or L1 distance

Minkowski Distance: The Flexible Parent

Minkowski distance is the general formula that includes both Euclidean and Manhattan as special cases

Formula: $D = (\Sigma|x_i - y_i|^p)^{(1/p)}$

- When **p = 1**: You get Manhattan distance
- When **p = 2**: You get Euclidean distance
- When **p = 3, 4, 5...**: You get higher-order distances (rarely used)
  Think of p as a "tuning knob" that lets you adjust how the distance is calculated. Most of the time, we stick with p=1 or p=2

Other Distance Metrics

- **Cosine Distance**: Measures the angle between two vectors rather than their magnitude. Great for text analysis where you care about the direction (topic) more than the size (word count)

- **Hamming Distance**: Counts how many positions are different between two sequences. Perfect for categorical data like comparing DNA sequences or binary codes

- **Chebyshev Distance**: Takes the maximum difference across all dimensions. Less commonly used but helpful in specific game theory and optimization problems

Step 3: Find the k Nearest Neighbors

After calculating distances from your new fruit to all fruits in your training data, you sort these distances from smallest to largest. The k fruits with the smallest distances are your "k nearest neighbors"—these are the most similar fruits.

Example with k=5:
Let's say you calculated these distances:

- Apple #1: distance = 12.5
- Orange #1: distance = 45.2
- Apple #2: distance = 8.3
- Orange #2: distance = 15.7
- Apple #3: distance = 9.1
- Orange #3: distance = 52.8
- Apple #4: distance = 11.2

After sorting and picking the 5 closest:

1. Apple #2 (distance = 8.3)
2. Apple #3 (distance = 9.1)

3. Apple #4 (distance = 11.2)
4. Apple #1 (distance = 12.5)
5. Orange #2 (distance = 15.7)

Your 5 nearest neighbors are 4 apples and 1 orange.

Step 4: Make a Prediction

Now comes the final step: using these neighbors to predict the class of your new fruit. The approach differs based on whether you're doing **classification** or **regression**

For Classification: Majority Voting

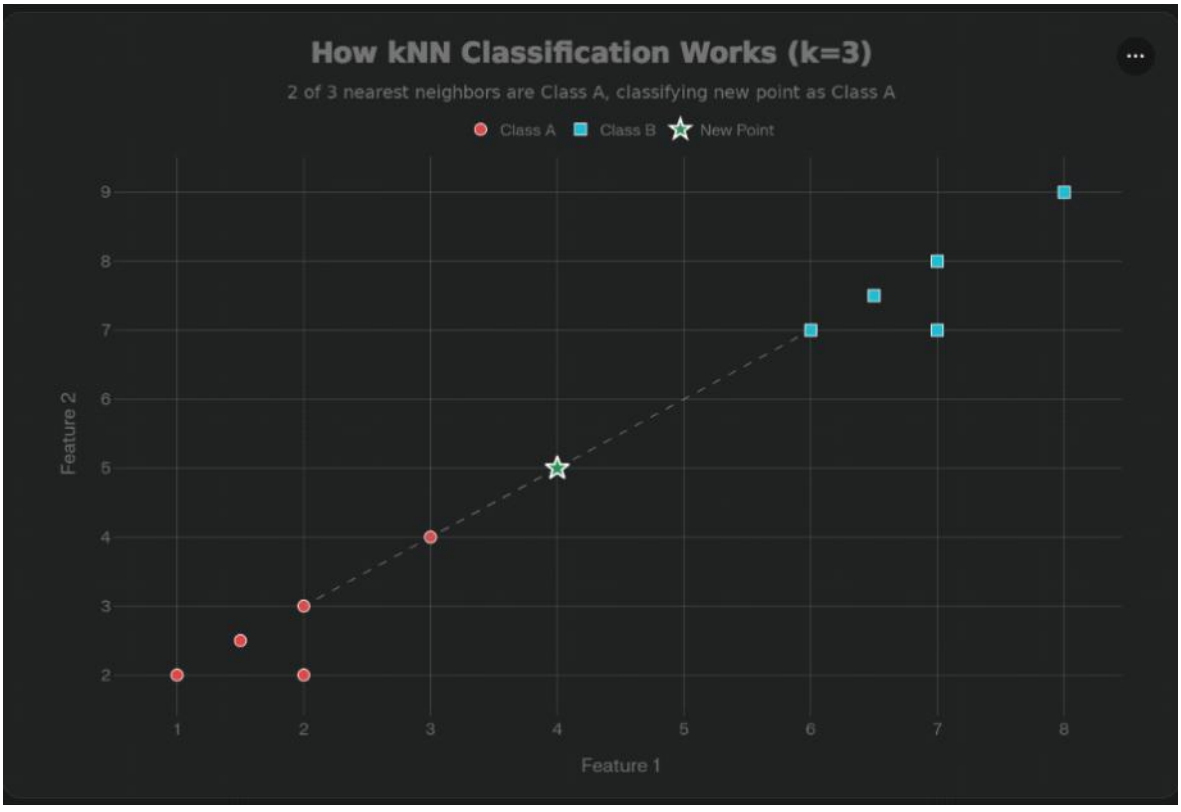
Count up the classes of your k neighbors and pick the most common one. In our fruit example, you have 4 apples and 1 orange, so the majority vote is **"apple"**. Your new fruit is classified as an apple!
This is exactly like taking a vote among your friends—whatever most people say wins

For Regression: Taking the Average

When predicting a continuous number (like house prices or temperature), you average the values of your k neighbors.

Example: You want to predict the price of a house. Your 5 nearest neighbors are houses that sold for: $250K, $275K, $260K, $255K, and $270K.
Average = (250 + 275 + 260 + 255 + 270) / 5 = **$262K**
Your prediction is $262,000

Visual illustration of kNN classification with k=3. The green star represents a new data point, and its classification is determined by the majority vote of its 3 nearest neighbors (2 from Class A, 1 from Class B), resulting in Class A prediction



Advanced Concept: Weighted kNN

Sometimes not all neighbors should have equal say. A fruit that's *very* close (distance = 2) should probably influence your decision more than one that's farther away (distance = 15), even if both are in the k nearest neighbors.
Weighted kNN assigns higher importance to closer neighbors using a weight based on distance. A common weighting scheme is:

Weight = 1 / distance

So a neighbor at distance 2 gets weight 1/2 = 0.5, while a neighbor at distance 10 gets weight 1/10 = 0.1

For classification, instead of simple majority voting, you sum up the weights for each class. For regression, you take a weighted average. This makes predictions more accurate and less sensitive to the choice of k.

The Importance of Feature Scaling

Here's a critical detail that can make or break your kNN model: **feature scaling**

Imagine you're comparing houses using two features:

- Size: ranges from 500 to 3,000 square feet
- Number of bedrooms: ranges from 1 to 5
  When calculating Euclidean distance:
- House A: 1,000 sq ft, 3 bedrooms
- House B: 1,500 sq ft, 4 bedrooms
  Distance = $\sqrt{[(1500-1000)^2 + (4-3)^2]} = \sqrt{[250,000 + 1]} = \sqrt{250,001} \approx 500$

Notice how the size difference (500) completely dominates the calculation, while the bedroom difference (1) barely matters! This is because the numbers for size are much larger

The solution: Scale all features to comparable ranges before applying kNN

Two Common Scaling Methods

1. Min-Max Scaling (transforms to range):

Formula: x_scaled = (x - min) / (max - min)

If sizes range from 500 to 3,000, a house with 1,000 sq ft becomes:
(1000 - 500) / (3000 - 500) = 500 / 2500 = 0.2

2. Standardization (centers around 0 with standard deviation 1):

Formula: x_scaled = (x - mean) / standard_deviation

This generally works well for most datasets and is often the preferred choice

Impact: In real experiments, kNN without feature scaling achieved 43.75% accuracy, while the same model with standardization achieved 62.5% accuracy—a massive 18% improvement just from scaling! Feature scaling ensures all features contribute fairly to distance calculations.

Lazy Learning vs. Eager Learning: A Philosophical Difference

kNN belongs to a category called **lazy learning** algorithms, which fundamentally differs from most other machine learning approaches

Lazy Learning (kNN)

- **Does not learn a model** during training
- Simply **stores all the training data** in memory
- When you ask for a prediction, it searches through all the stored data
- **Fast training** (instant—just store the data)
- **Slow prediction** (must calculate distances to all training points)
  Think of it like keeping every single textbook from school. When you need to answer a question, you search through all your books to find similar examples. No studying required, but answering takes time!

Eager Learning (Neural Networks, Decision Trees, SVM)

- **Builds a model** during training by finding patterns
- **Discards training data** after learning
- When you ask for a prediction, it applies the pre-built model
- **Slow training** (must process data and learn patterns)
- **Fast prediction** (just apply the formula)
  This is like studying hard before an exam, distilling all textbooks into notes. Studying takes time, but answering questions is quick!

Trade-offs:

- kNN uses more memory (stores everything) but adapts quickly to new data
- Eager learning uses less memory (just the model) but requires retraining for new patterns

Computational Complexity: The Performance Story

Understanding kNN's computational requirements helps you know when to use it

Time Complexity

For a single prediction with N training examples, d dimensions, and k neighbors:
O(N × d + N log k)

Breaking this down:

- **N × d**: Calculate distances to all N training points across d features
- **N log k**: Sort distances and find the k smallest ones

In simpler terms: If you have 10,000 training examples with 20 features, every single prediction requires 200,000 distance calculations! This is why kNN becomes slow

with large datasets.

Space Complexity
kNN stores the entire training dataset plus the distance matrix:
O(N_train × N_test)

For large datasets, this memory requirement can be prohibitive.

The Curse of Dimensionality: kNN's Achilles' Heel
As you add more and more features (dimensions) to your data, something strange happens: **kNN's performance deteriorates dramatically**. This phenomenon is called the **curse of dimensionality**.

Why This Happens
In high-dimensional spaces (many features), distances become meaningless:
1. **All points become equidistant**: Imagine you have 1,000 features. When calculating distances across so many dimensions, every data point starts looking roughly the same distance from every other point.
2. **Data becomes sparse**: To maintain the same density of data points, you need exponentially more data as dimensions increase. If you need 100 samples for 1 dimension, you'd need 100^2 = 10,000 for 2 dimensions, 100^3 = 1,000,000 for 3 dimensions, and so on.
3. **Nearest neighbors aren't really "near"**: Your k nearest neighbors might actually be quite far away in absolute terms, making their votes less meaningful.

Solutions to the Curse
1. **Dimensionality Reduction**: Use techniques like PCA (Principal Component Analysis) or t-SNE to reduce the number of features while preserving important information.
2. **Feature Selection**: Identify and keep only the most relevant features, discarding redundant or irrelevant ones.
3. **Use appropriate distance metrics**: Manhattan distance often performs better than Euclidean in high dimensions.

Handling Imbalanced Datasets
When one class vastly outnumbers another (like fraud detection where fraud is rare), kNN can struggle because majority voting naturally favors the majority class.

The Problem
Imagine classifying emails as spam (5% of data) or not spam (95% of data). If k=10, you might always find 9-10 "not spam" neighbors simply because they're so common, even if the email is actually spam.

Solutions
1. **Use smaller k values**: k=3 gives minority classes more chance to win votes
2. **Distance weighting**: Closer neighbors count more, reducing bias toward distant majority class members
3. **SMOTE (Synthetic Minority Over-sampling)**: Create synthetic examples of the minority class to balance the dataset
4. **Better evaluation metrics**: Use F1-score, precision, recall, or AUC-ROC instead of simple accuracy
5. **Stratified cross-validation**: Ensure each validation fold maintains the original class distribution

Classification vs. Regression: Two Sides of kNN
kNN handles both types of machine learning problems:
Classification
Goal: Assign a category label (e.g., apple vs. orange, spam vs. not spam)
Method: Majority voting among k neighbors
Example: Classifying iris flowers based on petal measurements—a classic machine learning dataset where kNN achieves 97.37% accuracy

Regression
Goal: Predict a continuous numerical value (e.g., house price, temperature)
Method: Average (or weighted average) of k neighbors' values
Example: Predicting house prices based on size, location, and number of rooms. If your 5 nearest neighbor houses sold for $250K, $275K, $260K, $255K, and $270K, your prediction is the average: $262K
The algorithm structure remains identical—only the final aggregation step changes.

Real-World Applications: Where kNN Shines
kNN's simplicity and effectiveness make it valuable across diverse industries:

Finance
- **Stock market prediction**: Analyzing company performance metrics to forecast

stock prices
- **Credit scoring**: Evaluating loan applicants based on similar historical cases
- **Fraud detection**: Identifying unusual transactions by comparing to normal patterns

Healthcare

- **Medical diagnosis**: Predicting diseases by comparing patient symptoms to historical cases
- **Gene expression analysis**: Classifying genetic patterns for research
- **Breast cancer prediction**: Using factors like mitosis levels to predict cancer symptoms

Recommendation Systems

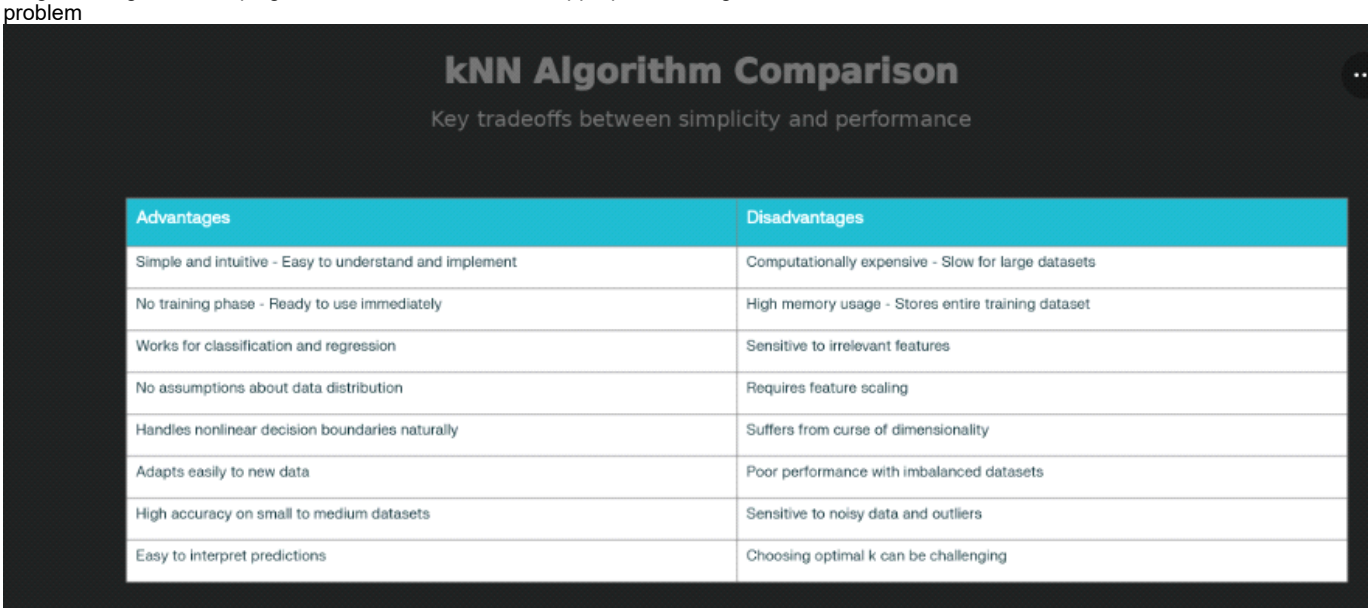
- **Netflix, Amazon, Hulu**: Recommending movies and products based on similar users' preferences
- **Content filtering**: Suggesting articles or videos aligned with user interests

Other Applications

- **Text mining and categorization**: Classifying documents into topics
- **Image and facial recognition**: Identifying objects and people in photos
- **Customer segmentation**: Grouping customers by purchasing behavior for targeted marketing
- **Agriculture**: Climate forecasting and soil analysis

Advantages and Disadvantages: The Complete Picture

Comprehensive comparison of the strengths and weaknesses of the k-Nearest Neighbors algorithm, helping to understand when kNN is appropriate for a given problem



Advantages	Disadvantages
Simple and intuitive - Easy to understand and implement	Computationally expensive - Slow for large datasets
No training phase - Ready to use immediately	High memory usage - Stores entire training dataset
Works for classification and regression	Sensitive to irrelevant features
No assumptions about data distribution	Requires feature scaling
Handles nonlinear decision boundaries naturally	Suffers from curse of dimensionality
Adapts easily to new data	Poor performance with imbalanced datasets
High accuracy on small to medium datasets	Sensitive to noisy data and outliers
Easy to interpret predictions	Choosing optimal k can be challenging

When to Use kNN

kNN excels when you have:

- Small to medium-sized datasets (thousands, not millions, of examples)
- Clean, well-labeled data
- Nonlinear decision boundaries that are hard to model with simpler algorithms
- Need for quick deployment without extensive training
- Problems where local patterns matter more than global patterns

When to Avoid kNN

Consider other algorithms when you have:

- Very large datasets (millions of examples)—prediction becomes too slow
- High-dimensional data (hundreds of features) without dimensionality reduction
- Severely imbalanced classes without adjustment techniques
- Real-time applications requiring instant predictions
- Extremely noisy data with many outliers

Practical Tips for Using kNN Successfully

1. **Always scale your features** before applying kNN—this is non-negotiable
2. **Use cross-validation to choose k**—don't guess! Try k values from 1 to 20 and plot accuracy
3. **Start with Euclidean distance**, but try Manhattan if you have high dimensions or outliers

4. **Remove irrelevant features** through feature selection to improve performance and speed
5. **Consider weighted kNN** to reduce sensitivity to the k value
6. **Use odd values of k** for binary classification to avoid ties (k=3, 5, 7 rather than k=2, 4, 6)
7. **Monitor for the curse of dimensionality**—if you have more than 20-30 features, apply dimensionality reduction
8. **For imbalanced data**, use stratified sampling, distance weighting, or SMOTE

Conclusion: The Beauty of Simplicity

The k-Nearest Neighbors algorithm embodies a profound truth in machine learning: sometimes the simplest ideas are the most powerful. By simply looking at similar examples and learning from them—exactly how humans often make decisions—kNN achieves impressive results across countless applications.

While it has limitations (computational intensity, curse of dimensionality, sensitivity to scale), understanding these challenges and applying appropriate solutions makes kNN a valuable tool in any machine learning toolkit. Its intuitive nature makes it perfect for learning machine learning concepts, while its effectiveness keeps it relevant in production systems worldwide

Whether you're classifying flowers, predicting house prices, recommending movies, or detecting fraud, kNN's principle remains elegantly simple: **ask your neighbors, and go with what most of them say**. Sometimes, that's all you need